Informe de Sprint 1

Antonio de Benito, Pablo Berástegui y Gonzalo García

Período: 21/01/2026 – 01/02/2026

Proyecto

TradeData – Análisis de Datos de Trading de Criptomonedas

1. Objetivo del Sprint

En este primer sprint vamos a, primero familiarizarnos con los datos de TradingView, y una vez hecho esto, descargarnos los datos de nuestra criptomoneda durante los últimos 4 años con una frecuencia de un día. Una vez hecho esto los datos fueron almacenados en un bucket de S3 en forma de csv.

La criptomoneda que nos fue asignada fue Ethereum y su símbolo era `ETHUSD`.

2. Historias de Usuario Incluidas

2.1. Historia de usuario 1

Como: Manager del proyecto
Quiero: Conocer el proyecto, la librería TradingView y descargarnos los datos
Para: Poder empezar el proyecto con garantías

2.1.1. Análisis

Primero nos descargamos el repositorio indicado, y una vez hecho esto nos descargamos los datos necesarios.

2.1.2. Implementación

En esta historia de usuario solo utilizamos el repositorio indicado y nos descargamos los `csv` de los datos indicados (4 últimos años).

2.1.3. Evidencias

Aquí vemos el código usado para descargar los CSVs

```python
eth_data = request.get_hist(
    symbol='ETHUSDT',
    exchange='BINANCE',
    interval=Interval.daily,
    n_bars=1460
)
for year, group in eth_data.groupby(eth_data.index.year):
    group.to_csv(f'eth_data_{year}.csv')
```

Figura 1: Código usado para descargar los CSVs.

2.1.4. Resultado

Aquí vemos los CSVs descargados

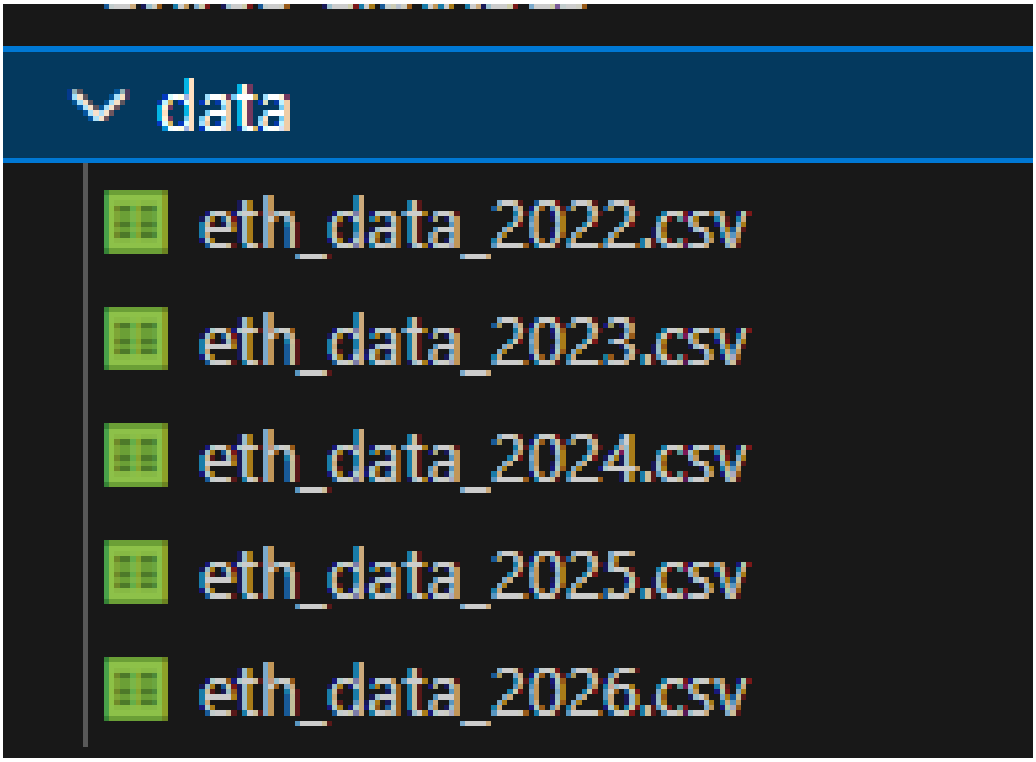

Figura 2: CSVs descargados.

2.2. Historia de usuario 2

Como: Manager del proyecto
Quiero: Diseñar una primera organización de la información basada en buckets de Amazon S3
Para: Empezar a definir la capa de datos históricos

2.2.1. Análisis

Para poder almacenar nuestros datos de forma ordenada hacemos uso de la herramienta S3.

2.2.2. Implementación

Analizamos el problema y llegamos a la conclusión de que la mejor idea era hacer un solo bucket de S3 con la información de todos los años. Esto nos permitirá hacer consultas sobre varios años de forma eficiente, ya que hemos considerado que esto puede ser un habitual en este proyecto. Para poder hacerlo todo desde código, aportamos nuestras claves de identificación para AWS mediante la terminal; de esta forma ha sido posible crear los buckets mediante código.

2.2.3. Evidencias

Aquí vemos el código usado para crear el bucket y subir los CSVs

```
s3.create_bucket(
    Bucket=bucket,
    CreateBucketConfiguration={
        "LocationConstraint": "eu-south-2"
    }
)

print("Bucket creado")

base_path = Path("data")

for csv_file in base_path.glob("eth_data_*.csv"):
    year = csv_file.stem.split("_")[-1]   # 2022, 2023, ...

    s3_key = f"eth/year={year}/{csv_file.name}"

    s3.upload_file(
        Filename=str(csv_file),
        Bucket=bucket,
        Key=s3_key
    )

    print(f"Subido {s3_key}")
```

Figura 3: Código usado para subir los CSVs y crear el bucket.

2.2.4. Resultado

Aqui vemos como se suben los csv.

```
(venv) berasteguip@PBM-zenbook:/mnt/c/3_imat/2_cuatrimestre/big_data/datos-grandes$ python s3_setup.py

Bucket creado
Subido eth/year=2022/eth_data_2022.csv
Subido eth/year=2023/eth_data_2023.csv
Subido eth/year=2024/eth_data_2024.csv
Subido eth/year=2025/eth_data_2025.csv
Subido eth/year=2026/eth_data_2026.csv
```

Figura 4: Datos subidos mediante codigo.

3. Arquitectura del Sprint

Primero nos encargamos de descargar los datos y luego creamos los buckets.

4. Validación y Pruebas

Una vez creada la arquitectura podemos acceder a nuestros datos ordenados en la nube.

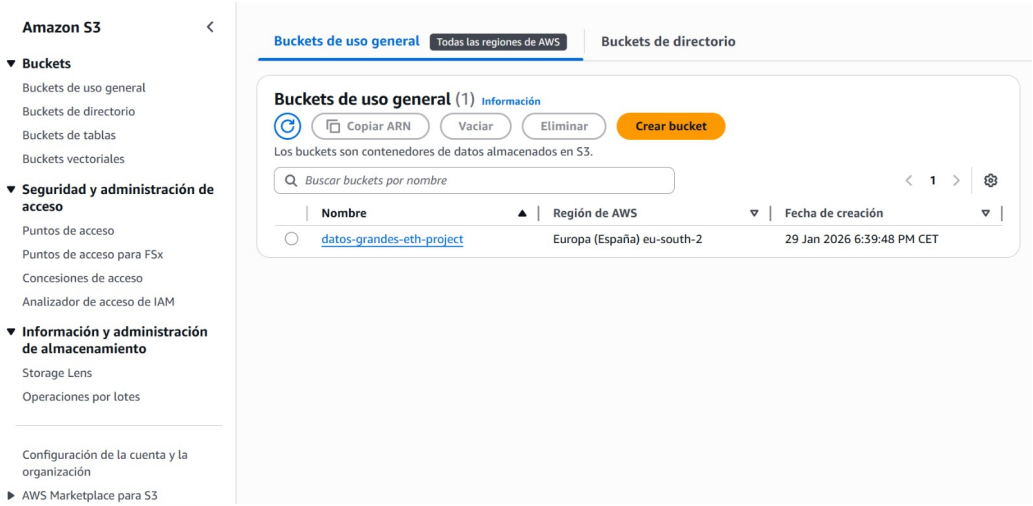

Figura 5: Bucket en la interfaz gráfica.

5. Conclusiones del Sprint

Una vez hecho esto ya tenemos todos nuestros datos organizados, de forma que podemos trabajar con ellos en próximos sprints.

6. Trabajo Pendiente y Próximo Sprint

Ahora queda poder hacer *queries* de forma sencilla sobre estos datos.

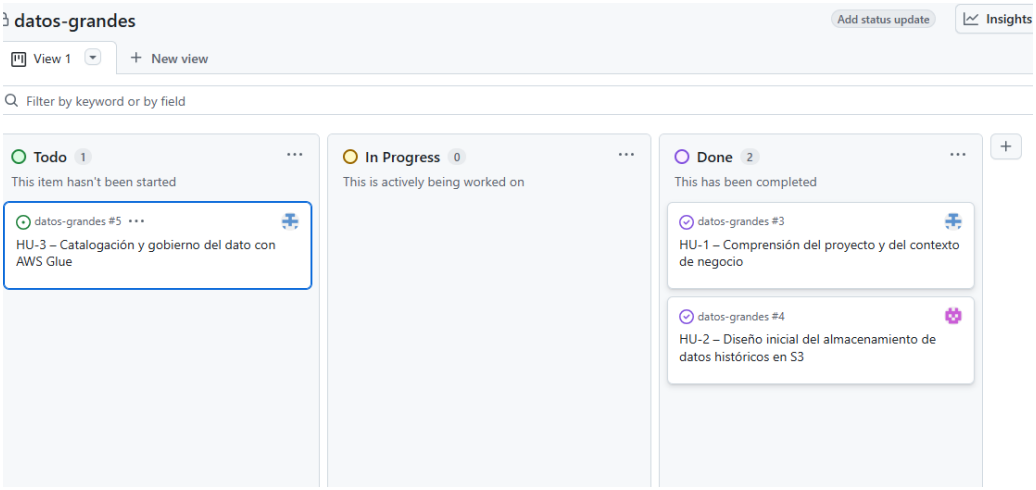

Figura 6: Tablero de gestión del proyecto, con HU completadas y siguientes.